Universidad Politécnica Internacional

Curso: técnicas de programación

Proyecto #1

Nutrición para todos

Documentación Final

Estudiante:

José Antonio Ramírez Calero

Profesor:

Luis Felipe Mora Umaña

I Cuatrimestre

31 de marzo del 2026

1. Introducción 3

2. Decisiones de diseño 4

3. División del proyecto 5

Epic1: Sistema de Nutrición para todos 5

Feature1.1: Gestión de usuarios 5

PBI 1.1.1: Registro de usuarios 5

PBI 1.1.2: Inicio de sesión 5

Feature1.2: Gestión de productos 6

PBI 1.2.1: Registro de productos 6

PBI 1.2.2: Edición de productos 6

Feature1.3: Gestión de menús 7

PBI 1.3.1: Crear menú..... 7

PBI 1.3.2: Modificar menú 8

PBI 1.3.3: Editar menú 8

PBI 1.3.4: Eliminar menú 9

Feature1.4: Información nutricional 10

PBI 1.4.1: Visualización de calorías de mantenimiento 10

PBI 1.4.2: Visualización de distribución de macronutrientes 10

PBI 1.4.3: Cálculo de IMC 11

Feature1.5: Estadísticas nutricionales 12

PBI 1.5.1: Consumo diario de calorías 12

PBI 1.5.2: Consumo diario de macronutrientes 12

PBI 1.5.3: Comparación con meta diaria 13

PBI 1.5.4: Conteo mensual de cumplimiento 13

PBI 1.5.5: Estadísticas parametrizables por fecha 14

PBI 1.5.6: Progreso del día 15

4. Desarrollo 15

5. Referencias 17

6. Bibliografía 18

1. Introducción

Desarrollo de sistema extensible de nutrición para la gestión de la alimentación de los usuarios.

El programa se desarrollará utilizando principios de programación, patrones de arquitectura, metodologías ágiles, herramientas de gestión de proyecto vistas en clases (POO, Clean Code, Modelo vista controlador, SCRUM, Jira, etc.)

El programa por desarrollar debe permitir la gestión de usuarios, manejo de menús, productos y estadísticas, de forma eficiente, intuitiva y organizada.

Este documento presenta las decisiones de diseño, la división del proyecto en épicas, features y PBIs, y una descripción del desarrollo propuesto para el sistema.

2. Decisiones de diseño

Para el desarrollo del sistema se contemplan arquitecturas, paradigmas de programación, patrones de diseño y practicas vistas en clase y sugeridas por el profesor.

Arquitectura del sistema: El sistema utilizará el patrón arquitectónico Modelo Vista Controlador (MVC), el cual permite separar las diferentes responsabilidades del sistema:

- **Modelo**: Maneja los datos y la lógica del sistema.
- **Vista**: Representa la interfaz gráfica con la que interactúa el usuario.
- **Controlador**: Gestiona la comunicación entre el modelo y la vista.

Además de MVC, el proyecto implementa una abstracción tipo Repository para desacoplar la lógica de negocio del mecanismo de persistencia. Esta decisión se materializa mediante la interfaz IRepository<T>, que define las operaciones de carga y guardado de entidades, y por medio de implementaciones concretas como UserRepository, ProductRepository, MenuRepository y MenuProductRepository.

Gracias a esta separación, los controladores trabajan contra contratos y no contra detalles específicos de archivos CSV, lo que mejora la mantenibilidad, la extensibilidad y la posibilidad de reemplazar la fuente de datos en futuras iteraciones sin afectar la lógica principal del sistema.

Paradigma de programación: Programación orientada a objetos.

Principios de programación: Se aplicarán principios de Solid y Clean Code para mantener un código claro, legible y fácil de mantener.

Metodología de trabajo: Para la organización del proyecto usaremos SCRUM para dividir las tareas y estructurar el proyecto.

3. División del proyecto

Epic1: Sistema de Nutrición para todos

Feature1.1: Gestión de usuarios

PBI 1.1.1: Registro de usuarios

Descripción: Como desarrollador, quiero que los usuarios tengas una ventana con un formulario, este formulario solicitara la información necesaria para que el usuario ingrese a la aplicación. Además, el objetivo y tipo de dieta de interés.

Criterios de Aceptación:

- El sistema genera una ventana con un formulario de registro de usuarios.
- El usuario puede ingresar nombre, correo, contraseña, objetivo y dieta de interés.
- El sistema valida que los campos no estén vacíos.

Tareas:

- Creación de clase usuario.
- Creación de ventana para el formulario de registro.
- Implementar validación de datos.

PBI 1.1.2: Inicio de sesión

Descripción: Como desarrollador, quiero que el usuario acceda a la aplicación mediante una validación del correo contraseña de su cuenta anteriormente creada en el registro.

Criterios de Aceptación:

- El sistema permite ingresar correo y contraseña.
- El sistema valida las credenciales del usuario**.**

Tareas:

- Crear ventana de login.
- Validar usuario y contraseña.
- Mostrar pantalla principal del sistema.

Feature1.2: Gestión de productos

PBI 1.2.1: Registro de productos

Descripción: Como desarrollador quiero permitir al usuario registrar productos alimenticios para utilizarlos en los menús, además el sistema contara con una base de 50 productos ya registrados.

Criterios de Aceptación:

- El sistema permite ingresar nombre del producto.
- El sistema permite ingresar información nutricional.

Tareas:

- Crear clase Producto.
- Crear formulario de registro de productos.
- Registrar base de 50 productos. • Guardar información nutricional.

PBI 1.2.2: Edición de productos

Descripción: Como desarrollador quiero permitir al usuario editar la información de los productos registrados.

Criterios de Aceptación:

- El sistema permite modificar información del producto.

Tareas:

- Crear función de edición.
- Actualizar datos del producto.

Feature1.3: Gestión de menús

PBI 1.3.1: Crear menú

Descripción: Como desarrollador, quiero que el usuario pueda crear un menú alimenticio para registrar los productos que consume durante el día, organizados por tiempos de comida.

Criterios de Aceptación:

- El sistema debe permitir crear un menú asociado a un usuario.
- El sistema debe permitir seleccionar una fecha para el menú.
- El sistema debe permitir dividir el menú por tiempos de comida, por ejemplo: desayuno, merienda, almuerzo y cena.
- El sistema debe permitir agregar productos o alimentos ya registrados en el sistema.
- El sistema debe permitir ingresar cantidades o porciones consumidas de cada producto.
- Al procesar el menú, el sistema debe calcular calorías, proteínas, carbohidratos y grasas totales.
- El menú debe quedar almacenado en el sistema.

Tareas:

- Crear clase menú.
- Crear clase tiempo de comida.
- Crear relación entre menú y producto.
- Crear relación entre usuario y menú.
- Diseñar formulario para crear menú.
- Permitir selección de fecha.

- Permitir agregar alimentos por tiempo de comida.
- Implementar cálculo de calorías y macronutrientes.
- Guardar menú en el sistema.

PBI 1.3.2: Modificar menú

Descripción: Como desarrollador, quiero permitir al usuario modificar los productos o cantidades dentro de un menú ya creado, para poder ajustar la información de los alimentos consumidos durante el día.

Criterios de Aceptación:

- El sistema permite seleccionar un menú previamente creado.
- El sistema permite agregar nuevos productos al menú.
- El sistema permite modificar las cantidades o porciones de los productos agregados.
- El sistema permite eliminar productos específicos del menú.
- El sistema recalcula automáticamente calorías, proteínas, carbohidratos y grasas al modificar el menú.
- El sistema guarda los cambios realizados en el menú.

Tareas:

- Permitir selección de menú existente.
- Crear función para agregar productos adicionales.
- Permitir modificar cantidades de productos.
- Permitir eliminar productos del menú.
- Recalcular valores nutricionales automáticamente.
- Guardar cambios realizados en el menú.

PBI 1.3.3: Editar menú

Descripción: Como desarrollador, quiero permitir al usuario editar la información general de un menú previamente creado, como la fecha o los tiempos de comida, para mantener organizada la información del consumo diario.

Criterio de Aceptación:

- El sistema permite seleccionar un menú existente para editar.

- El sistema permite modificar la fecha del menú.
- El sistema permite modificar los tiempos de comida asociados.
- El sistema mantiene los productos previamente registrados en el menú.
- El sistema guarda los cambios realizados.

Tareas:

- Crear función para editar menú existente.
- Permitir modificación de fecha del menú.
- Permitir modificar tiempos de comida.
- Mantener relación entre menú y productos.
- Guardar cambios del menú editado.

PBI 1.3.4: Eliminar menú

Descripción: Como desarrollador, quiero permitir al usuario eliminar un menú previamente registrado, para remover información incorrecta o que ya no sea necesaria dentro del sistema.

Criterio de Aceptación:

- El sistema permite seleccionar un menú registrado.
- El sistema solicita confirmación antes de eliminar el menú.
- El menú eliminado ya no aparece en la lista de menús del usuario.
- El sistema actualiza las estadísticas relacionadas con los menús.

Tareas:

- Crear función para eliminar menú.
- Permitir selección de menú existente.
- Implementar mensaje de confirmación antes de eliminar.
- Eliminar menú de la base de datos.
- Actualizar lista de menús del usuario.

Feature1.4: Información nutricional

PBI 1.4.1: Visualización de calorías de mantenimiento

Descripción: Como desarrollador, quiero que el sistema muestre al usuario sus calorías de mantenimiento según sus datos personales y su objetivo, para que pueda tener una referencia nutricional adecuada.

Criterios de Aceptación:

- El sistema muestra las calorías de mantenimiento del usuario.
- El cálculo considera información del usuario como peso, altura, edad, sexo y objetivo.
- El sistema permite mostrar variaciones según el objetivo del usuario, por ejemplo: mantenimiento, pérdida o aumento de peso.
- La información se muestra en una pestaña de información nutricional.

Tareas:

- Crear función para calcular calorías de mantenimiento.
- Registrar y utilizar datos físicos del usuario.
- Relacionar cálculo con el objetivo del usuario.
- Diseñar pestaña de información nutricional.
- Mostrar resultado de calorías de mantenimiento.

PBI 1.4.2: Visualización de distribución de macronutrientes

Descripción: Como desarrollador, quiero que el sistema muestre la distribución de macronutrientes del usuario, para que pueda conocer cuántas proteínas, carbohidratos y grasas debería consumir según su objetivo.

Criterios de Aceptación:

- El sistema muestra la distribución de proteínas, carbohidratos y grasas.

- La distribución se calcula en función de las calorías y del objetivo del usuario.
- El sistema muestra la información en gramos o porcentajes.
- La información se presenta en la pestaña de información nutricional.

Tareas:

- Crear función para calcular distribución de macronutrientes.
- Relacionar cálculo con calorías de mantenimiento.
- Relacionar cálculo con objetivo del usuario.
- Mostrar proteínas, carbohidratos y grasas en pantalla.
- Integrar resultados en pestaña de información.

PBI 1.4.3: Cálculo de IMC

Descripción: Como desarrollador, quiero que el sistema calcule y muestre el índice de masa corporal del usuario, para brindarle una referencia básica sobre su estado físico.

Criterios de Aceptación:

- El sistema calcula el IMC utilizando peso y altura del usuario.
- El sistema muestra el resultado del IMC.
- El sistema muestra la categoría correspondiente del IMC.
- El resultado se presenta en la pestaña de información nutricional.

Tareas:

- Crear función para calcular IMC.
- Utilizar peso y altura registrados del usuario.
- Mostrar resultado del cálculo.
- Mostrar categoría correspondiente del IMC.
- Integrar resultado en la pestaña de información.

Feature1.5: Estadísticas nutricionales

PBI 1.5.1: Consumo diario de calorías

Descripción: Como desarrollador, quiero que el sistema muestre el consumo diario de calorías del usuario, para que pueda comparar lo consumido con su meta nutricional diaria.

Criterios de Aceptación:

- El sistema muestra las calorías consumidas en el día.
- El sistema compara las calorías consumidas con la meta diaria del usuario.
- El sistema permite consultar esta información por fecha.
- La información se actualiza según los menús registrados por el usuario.

Tareas:

- Crear función para sumar calorías consumidas en el día.
- Relacionar consumo diario con menús registrados.
- Comparar calorías consumidas con meta diaria.
- Implementar filtro por fecha.
- Mostrar resultado en pantalla.

PBI 1.5.2: Consumo diario de macronutrientes

Descripción: Como desarrollador, quiero que el sistema muestre el consumo diario de proteínas, carbohidratos y grasas del usuario, para que pueda conocer su avance respecto a su meta nutricional.

Criterios de Aceptación:

- El sistema muestra proteínas, carbohidratos y grasas consumidas en el día.
- El sistema compara los macronutrientes consumidos con la meta diaria del usuario.
- El sistema permite consultar la información por fecha.
- La información se actualiza con base en los menús registrados.

Tareas:

- Crear función para sumar macronutrientes diarios.
- Comparar consumo de macros con meta diaria.
- Implementar filtro por fecha.
- Mostrar resultados por macronutriente.
- Actualizar estadísticas según menús registrados.

PBI 1.5.3: Comparación con meta diaria

Descripción: Como desarrollador, quiero que el sistema compare el consumo diario del usuario con su meta nutricional, para indicarle cuánto le falta o cuánto ha excedido en calorías y macronutrientes.

Criterios de Aceptación:

- El sistema compara calorías consumidas con calorías meta.
- El sistema compara proteínas, carbohidratos y grasas consumidas con la meta diaria.
- El sistema indica cuánto falta o cuánto se excedió.
- La comparación puede consultarse por fecha.

Tareas:

- Crear función para comparar consumo con meta diaria.
- Calcular faltante o excedente de calorías.
- Calcular faltante o excedente de macronutrientes.
- Implementar parámetro de consulta por fecha.
- Mostrar comparación en pantalla.

PBI 1.5.4: Conteo mensual de cumplimiento

Descripción: Como desarrollador, quiero que el sistema genere un conteo mensual de cumplimiento de metas nutricionales, para que el usuario pueda conocer cuántos días logró cumplir con su objetivo.

Criterios de Aceptación:

- El sistema permite consultar estadísticas por mes o rango de fechas.
- El sistema muestra cuántos días se cumplió la meta diaria.
- El sistema muestra cuántos días no se cumplió la meta diaria.

- El sistema compara los días cumplidos con los días no cumplidos.

Tareas:

- Crear función para contar días de cumplimiento.
- Crear función para contar días sin cumplimiento.
- Implementar consulta mensual.
- Implementar consulta por rango de fechas.
- Mostrar resultados comparativos.

PBI 1.5.5: Estadísticas parametrizables por fecha

Descripción: Como desarrollador, quiero que el sistema permita consultar estadísticas nutricionales por fechas, para que el usuario pueda analizar su progreso en distintos períodos de tiempo.

Criterio de Aceptación:

- El sistema permite seleccionar una fecha específica.
- El sistema permite seleccionar un rango de fechas.
- El sistema genera estadísticas con base en la fecha o rango seleccionado.
- El sistema muestra resultados actualizados según el parámetro indicado.

Tareas:

- Crear filtro por fecha.
- Crear filtro por rango de fechas.
- Relacionar estadísticas con fechas de menús. • Actualizar cálculos según fecha seleccionada.
- Mostrar resultados filtrados en pantalla.

PBI 1.5.6: Progreso del día

Descripción: Como desarrollador, quiero que el sistema muestre al usuario cuánto le falta durante el día para llegar a su meta nutricional, para que pueda ajustar su alimentación antes de finalizar la jornada.

Criterios de Aceptación:

- El sistema muestra cuánto falta para alcanzar la meta calórica diaria.

- El sistema muestra cuánto falta para alcanzar la meta de proteínas, carbohidratos y grasas.
- La información se actualiza en tiempo de visualización.
- El progreso se calcula con base en los alimentos registrados durante el día.

Tareas:

- Crear función para calcular progreso diario.
- Calcular faltante de calorías.
- Calcular faltante de macronutrientes.
- Actualizar valores en tiempo de visualización.
- Mostrar progreso del día en pantalla.

4. Desarrollo

Como utilizar y ejecutar el programa, interacciones con el programa, análisis de resultados, aprendizajes y conclusiones.

El sistema Nutrición para todos se desarrolla como una aplicación orientada a facilitar el registro y análisis de la alimentación diaria de los usuarios. Su objetivo principal es permitir que cada usuario pueda registrar los alimentos que consume, analizar su información nutricional y comparar su consumo con metas nutricionales previamente definidas.

El repositorio del proyecto contiene tanto el código fuente del sistema como la documentación técnica correspondiente, con el fin de facilitar su comprensión, ejecución y mantenimiento.

4.2 Componentes principales implementados

Dentro del desarrollo final del proyecto se implementaron los siguientes componentes principales:

Gestión de usuarios: registro, inicio de sesión y manejo del usuario autenticado.

Gestión de productos: creación y edición de productos con información nutricional.

Gestión de menús: creación, modificación, edición y eliminación de menús asociados al usuario.

Información nutricional: cálculo de calorías de mantenimiento, meta nutricional, distribución de macronutrientes e IMC.

Estadísticas nutricionales: cálculo de consumo diario, diferencia contra metas, progreso del día y consultas por fecha o rango.

4.3 Persistencia de datos

- Durante esta primera iteración se utilizó persistencia basada en archivos CSV para simplificar la implementación y mantener una estructura legible. Los archivos utilizados son:
- data/users.csv
- data/products.csv
- data/menus.csv
- data/menuProducts.csv

Al cierre de esta documentación, el sistema cuenta con:

- 27 usuarios registrados
- 50 productos registrados
- 393 menús almacenados
- 1563 registros de comidas

La persistencia se maneja mediante repositorios que implementan la interfaz IRepository<T>, lo cual separa el acceso a datos de la lógica de negocio del sistema.

4.4 Ejecución del programa

Para ejecutar el sistema es necesario contar con un proyecto de visual estudio y clonar el repositorio alojado en esta dirección de GitHub: https://github.com/Jcaleriux/TP-P1-Nutricion.git.

Una vez clonado el repositorio, el usuario debe abrir el proyecto en el entorno de desarrollo y ejecutar el archivo principal de la aplicación.

Análisis de resultados, aprendizajes y conclusiones

El desarrollo del sistema Nutrición para todos permite aplicar de forma práctica diversos conceptos vistos en clase. A lo largo de esta primera fase aprendí sobre Jira, SCRUM y aplique conocimientos anteriores para el desarrollo de cada feature y PBI. Esta metodología permitió identificar claramente las funcionalidades necesarias para el sistema y establecer un orden lógico en su implementación.

Durante el desarrollo del proyecto obtuve habilidades relacionadas con el análisis de requisitos, la planificación del desarrollo de software y la organización de funcionalidades mediante metodologías ágiles.

En conclusión, el proyecto representa una aplicación práctica de los conocimientos adquiridos en el curso y demuestra cómo los principios de diseño y organización del software pueden utilizarse para desarrollar soluciones que faciliten el manejo de información y el análisis de datos en un contexto real.

5. Diagrama de Gantt

Actividad	**Sem 8**	**Sem 9**	**Sem 10**	**Sem 11**	**Sem 12**	**Sem 13**
Análisis y organización del proyecto	X					
Elaboración del backlog (Epic, Features, PBIs)	X					
Diseño de clases y estructura MVC	X	X				
Implementación de gestión de usuarios		X	X			

Implementación de gestión de productos		X	X			
Implementación de creación de menús			X	X		
Implementación de modificación, edición y eliminación de menús				X	X	
Implementación de información nutricional				X	X	
Implementación de estadísticas nutricionales					X	X
Integración general del sistema						X

6. Referencias

Como clonar un repositorio. Desde: Clonar un repositorio - Documentación de GitHub

Como añadir archivos a GitHub. Desde: Adding a file to a repository - GitHub Docs

Introducción a Jira. Desde: Introducción a Jira: guía completa para principiantes | Atlassian

Video de Gestion de Scrum en Jira. Desde: Gestiona tu proyecto SCRUM en JIRA paso a paso

Patrones de diseño. Desde: El catálogo de patrones de diseño

Que es MVC. Desde: El patrón modelo-vista-controlador: Arquitectura y frameworks explicados

Microsoft .NET documentation. Microsoft Learn. Desde: https://learn.microsoft.com/dotnet/

Repository Design Pattern - GeeksforGeeks Desde: https://www.geeksforgeeks.org/system-design/repository-design-pattern/